

Plataforma de MÉTODOS NUMÉRICOS

Manual Técnico



C# / .NET API
Backend & REST



React + TypeScript
Interfaz de Usuario



Python Worker
Motor de Cálculo



BD(SQLserver)
persistencia de datos

Tabla de Contenidos

1. Descripción General del Sistema
2. Arquitectura del Sistema
3. Descripción de Componentes
4. Modelo de Base de Datos
5. Endpoints REST de la API
6. Flujo de Ejecución de un Job
7. Justificación de Decisiones Técnicas
8. Configuración de Variables de Entorno
9. Casos de Error y Manejo de Excepciones
10. Estructura del Repositorio

1. Descripción General del Sistema

El presente proyecto consiste en una plataforma web de ejecución, trazabilidad y comparación de métodos numéricos. El sistema permite al usuario configurar un Job seleccionando un método numérico ya sea de búsqueda de raíces o de resolución de sistemas lineales junto con sus parámetros de entrada, y observar la ejecución asíncrona iteración por iteración a través de una interfaz gráfica moderna. El resultado final incluye la solución calculada, la tabla completa de iteraciones y una gráfica de convergencia del error relativo.

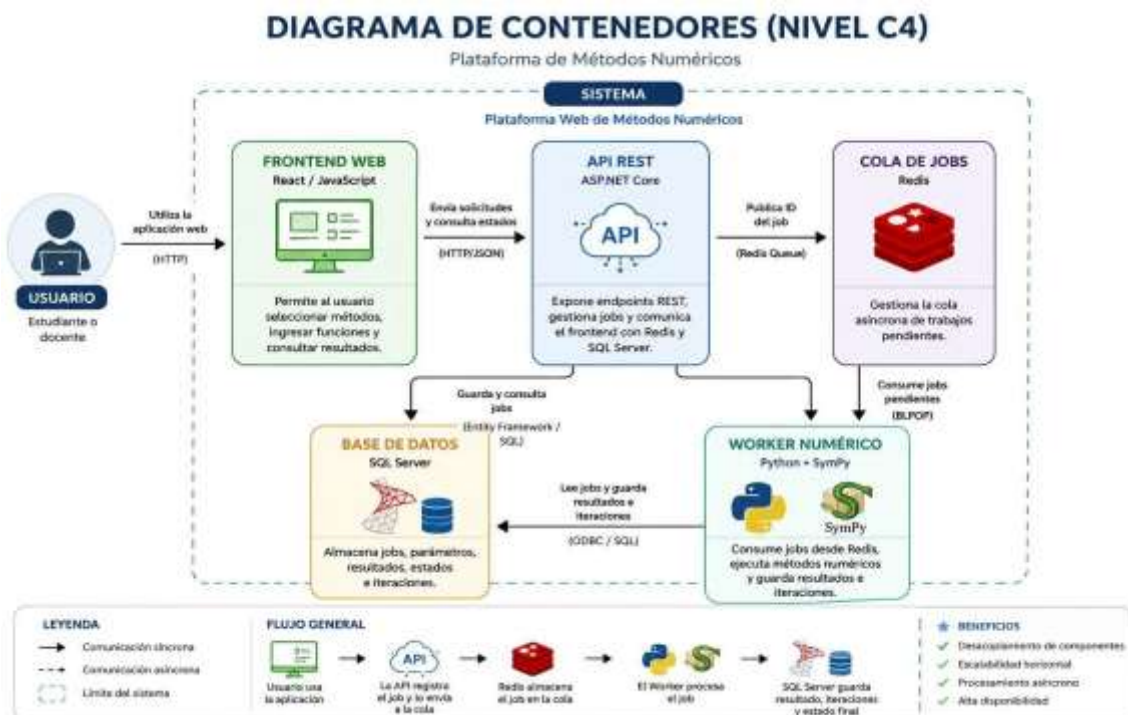
El sistema fue desarrollado utilizando una arquitectura híbrida empresarial que desacopla la interfaz de usuario, la API principal, el motor de cómputo numérico y la capa de persistencia en servicios independientes coordinados mediante una cola de mensajes. Todos los servicios se distribuyen como contenedores Docker y pueden levantarse con un único comando: Docker compose up --build.

2. Arquitectura del Sistema

La arquitectura del sistema sigue el patrón de arquitectura por contenedores compuesto por cinco servicios independientes que se comunican a través de interfaces bien definidas. A continuación, se describe el diagrama de contenedores (nivel C4).

2.1 Vista de Contenedores (C4 – Container Level)

El siguiente diagrama muestra los cinco contenedores que forman el sistema y sus responsabilidades, junto con el flujo de comunicación entre ellos



2.2 Descripción del Flujo entre Contenedores

El flujo de comunicación entre contenedores sigue el siguiente orden de ejecución:

1. El usuario interactúa con el Frontend SPA (navegador web).
2. El Frontend envía POST /jobs a la API .NET con método y parámetros en formato JSON.
3. La API valida el request e inserta el job en SQL Server con status = PENDING, generando un jobId único.
4. Con el jobId confirmado en base de datos, la API ejecuta RPUSH job_queue {jobId} en Redis.
5. El Worker Python, bloqueado con BLPOP job_queue, recibe el jobId de Redis.
6. El Worker consulta SQL Server para obtener parámetros y actualiza status = RUNNING.
7. El Worker ejecuta el método numérico iteración por iteración dentro de su propio proceso.
8. Al finalizar, el Worker inserta las iteraciones en JobIterations y actualiza el job a DONE o FAILED.
9. La API responde al Frontend con HTTP 201 Created y el jobId.
10. El Frontend detecta el cambio de estado mediante polling cada 5 s y renderiza resultados.

Nota: El paso 3 (INSERT en SQL Server) siempre ocurre antes del paso 4 (RPUSH en Redis). Este orden es crítico para garantizar que el Worker encuentre el registro en la base de datos cuando lo busque.

3. Descripción de Componentes

3.1 Frontend (SPA)

La interfaz de usuario es una Single Page Application construida con React 19, TypeScript y Vite 6. Tailwind CSS se encarga de los estilos. La biblioteca Recharts provee las gráficas de convergencia (error vs. iteración), y KaTeX realiza el renderizado LaTeX de las expresiones matemáticas ingresadas por el usuario. El frontend implementa polling automático cada cinco segundos sobre GET /jobs/{id} para detectar cambios de estado del job.

Responsabilidades:

- Formulario de creación de job con selección de método y parámetros.
- Validación de entradas del lado del cliente antes de enviar el request.
- Visualización del estado en tiempo real (PENDING, RUNNING, DONE, FAILED).
- Gráfica de convergencia y tabla de iteraciones al finalizar el job.
- Renderizado LaTeX de la función ingresada.

3.2 API Principal (.NET 10)

La API principal está construida sobre ASP.NET Core (.NET 10) y expone una interfaz REST documentada con OpenAPI/Swagger mediante la biblioteca Scalar. La persistencia se realiza a través de Entity Framework Core 9, que mapea las entidades Job y JobIteration al esquema relacional en SQL Server. La comunicación con Redis se gestiona mediante la biblioteca StackExchange.Redis. La API no ejecuta ninguna lógica numérica; su responsabilidad es exclusivamente la orquestación, validación y persistencia.

La estructura interna de la API sigue la organización: Controllers (exposición HTTP), Services (lógica de negocio), DTOs (contratos de request/response), Models (entidades EF Core), Data (DbContext y migrations).

3.3 Worker Numérico (Python 3.12)

El Worker es un proceso Python de larga duración que escucha la cola de Redis. Cuando recibe un jobId, consulta los parámetros del job en SQL Server, selecciona el método correspondiente mediante un dispatcher y ejecuta el algoritmo numérico iteración por iteración. La biblioteca SymPy se utiliza únicamente para el cálculo simbólico de derivadas en Newton-Raphson cuando el usuario omite la derivada. NumPy se permite para operaciones matriciales auxiliares. No se utiliza ninguna librería que resuelva el método completo.

La estructura interna del worker es: methods/ (un archivo por método), config.py (parámetros de conexión leídos de variables de entorno), db.py (acceso a SQL Server mediante pyodbc),

redis_client.py (cliente Redis), dispatcher.py (router de métodos), main.py (bucle principal BLPOP).

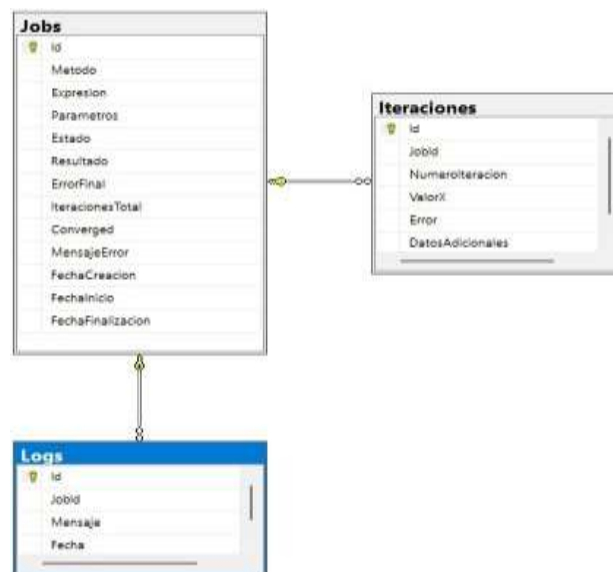
3.4 Cola de Mensajes (Redis 7)

Redis actúa exclusivamente como mecanismo de desacoplamiento asíncrono entre la API y el Worker. La API inserta jobIds mediante RPush job_queue y el Worker los consume mediante BLPOP job_queue. Redis almacena únicamente el identificador del job, nunca los parámetros completos, los cuales permanecen en SQL Server. La idempotencia se garantiza verificando el estado del job antes de ejecutarlo: si ya es DONE o RUNNING, se descarta.

3.5 Base de Datos (SQL Server 2022)

SQL Server 2022 provee la persistencia relacional del sistema. El esquema contiene dos tablas principales: Jobs y Iteraciones. La inicialización del esquema se realiza mediante scripts SQL ubicados en la carpeta sql-init/, que son ejecutados automáticamente al arrancar el contenedor.

4. Modelo de Base de Datos



4.1 Tabla: Jobs

Almacena la información principal de cada trabajo de ejecución numérica.

Columna	Tipo	Descripción
Id	INT (PK)	Identificador único del job
Metodo	NVARCHAR	Nombre del método numérico
Expresion	NVARCHAR	Expresión matemática a evaluar
Parametros	NVARCHAR(MAX)	Parámetros en formato JSON
Estado	NVARCHAR(20)	Estado: PENDING, RUNNING, DONE, FAILED
Resultado	NVARCHAR(MAX)	Resultado final del método
ErrorFinal	FLOAT	Error final del método
IteracionesTotal	INT	Número total de iteraciones ejecutadas
Converged	BIT	Indica si el método convergió
MensajeError	NVARCHAR(MAX)	Mensaje de error en caso de fallo
FechaCreacion	DATETIME2	Fecha y hora de creación
FechaInicio	DATETIME2	Fecha y hora de inicio de ejecución
FechaFinalizacion	DATETIME2	Fecha y hora de finalización

4.2 Tabla: Iteraciones

Almacena el detalle de cada iteración ejecutada por el Worker para un job determinado.

Columna	Tipo	Descripción
Id	INT (PK)	Identificador único de la iteración
JobId	INT (FK → Jobs.Id)	Referencia al job padre
NumeroIteracion	INT	Número secuencial de la iteración
ValorX	NVARCHAR(MAX)	Valor xi calculado (o vector x para sistemas lineales)
Error	FLOAT	Error relativo en la iteración
DatosAdicionales	NVARCHAR(MAX)	Datos adicionales específicos del método (JSON)

4.3 Relaciones y Restricciones

La tabla Iteraciones tiene una relación de clave foránea (FK) hacia Jobs.Id con eliminación en cascada. El campo Parametros de Jobs almacena un objeto JSON con los parámetros específicos de cada método, evitando la necesidad de columnas dispersas. El campo Datos adicionales de Iteraciones almacena información extra dependiente del método (por ejemplo, $f(x_i)$ y $f'(x_i)$ para Newton-Raphson).

5. Endpoints REST de la API

La API expone los siguientes endpoints REST. Todos están documentados y probables desde Swagger en <http://localhost:5000/swagger>.

Verbo	Ruta	Descripción	Prioridad
POST	/jobs	Crear un nuevo job (método + parámetros).	Core
GET	/jobs	Listar jobs. Soporta filtros: ?status=Done&method=newton-raphson.	Core
GET	/jobs/{id}	Detalle, estado y resultado de un job.	Core
GET	/jobs/{id}/iterations	Historial completo de iteraciones del job.	Core

5.1 Contrato de Request/Response

La API utiliza DTOs explícitos para todos los endpoints, sin exponer directamente las entidades de Entity Framework Core. El body del POST /jobs tiene la siguiente estructura:

```
{ "method": "newton-raphson", "function": "x^3 - x - 2", "x0": 1.5,  
  "tolerance": 1e-6, "maxIterations": 100 }
```

La respuesta de GET /jobs/{id} incluye los campos: id, method, status, result, converged, errorMessage, createdAt, startedAt, finishedAt y executionTimeMs.

6. Flujo de Ejecución de un Job

El ciclo de vida de un job sigue el siguiente flujo de estados sin saltos ni reversiones:

PENDING → RUNNING → DONE / FAILED

El estado PENDING se asigna al momento de la creación del job por parte de la API. El estado RUNNING se asigna cuando el Worker toma el job de la cola e inicia la ejecución. El estado DONE indica que el método terminó exitosamente (puede incluir converged = false si se alcanzó el máximo de iteraciones). El estado FAILED indica que ocurrió un error irreparable durante la ejecución (división por cero, matriz singular, expresión inválida, etc.).

La idempotencia del sistema garantiza que un job en estado DONE o RUNNING no sea re-ejecutado aunque su jobId vuelva a aparecer en la cola de Redis.

7. Justificación de Decisiones Técnicas

7.1 ¿Por qué Redis como cola de mensajes?

Redis fue seleccionado por su sencillez operacional, baja latencia y soporte nativo de estructuras tipo lista con operaciones bloqueantes (BLPOP). El mecanismo BLPOP permite al Worker esperar de forma eficiente nuevos trabajos sin polling activo, reduciendo el consumo de CPU. Además, Redis permite escalar horizontalmente el número de Workers sin modificar la API, ya que todos comparten la misma lista.

7.2 ¿Por qué Python para el Worker numérico

Python fue elegido para el Worker por la disponibilidad de SymPy, que permite realizar diferenciación simbólica automática de expresiones matemáticas arbitrarias, funcionalidad esencial para el método de Newton-Raphson cuando el usuario omite la derivada. Asimismo, el ecosistema matemático de Python (NumPy para operaciones matriciales) facilita la implementación de los métodos de sistemas lineales. Python permite además una implementación más natural y legible de los algoritmos numéricos iterativos.

7.3 ¿Por qué arquitectura híbrida (.NET + Python)?

La separación entre la API en .NET y el Worker en Python responde a criterios de especialización tecnológica: .NET 10 ofrece rendimiento superior para la capa de orquestación HTTP, validación de contratos REST y acceso a base de datos mediante Entity Framework Core, mientras que Python es el lenguaje más adecuado para el cómputo numérico simbólico. Esta separación también permite escalar cada capa de forma independiente y aislar fallos: un crash del Worker no afecta la disponibilidad de la API.

7.4 ¿Por qué SQL Server como base de datos?

SQL Server 2022 provee garantías ACID para las operaciones de escritura de jobs e iteraciones, lo cual es crítico para mantener la consistencia del ciclo de vida PENDING → RUNNING → DONE/FAILED. Su soporte para tipos JSON en columnas NVARCHAR permite almacenar los parámetros variables de cada método sin necesidad de tablas adicionales.

8. Configuración de Variables de Entorno

Todas las variables de configuración sensibles se gestionan a través de un archivo `.env` en la raíz del repositorio. El archivo `.env.example` contiene los nombres de todas las variables requeridas sin valores reales. Las variables son inyectadas a los contenedores a través de `docker-compose.yml`.

Variable	Servicio que la usa	Descripción
SQL_PASSWORD	api, worker, sqlserver	Contraseña del usuario de SQL Server
SQL_USER	api, worker	Usuario de SQL Server
SQL_DATABASE	worker	Nombre de la base de datos
REDIS_HOST	worker	Hostname del servicio Redis (redis)
REDIS_PORT	worker	Puerto de Redis (6379)
REDIS_DB	worker	Base de datos Redis (0)
REDIS_QUEUE	worker	Nombre de la lista de Redis (job_queue)
ASPNETCORE_ENVIRONMENT	api	Entorno de ASP.NET Core (Development)

9. Casos de Error y Manejo de Excepciones

El Worker captura todos los casos de error sin colapsar el proceso. Cuando ocurre un error durante la ejecución de un método, el Worker actualiza el job a `status = FAILED` e inserta un mensaje descriptivo en el campo `errorMessage`. El Worker continúa en ejecución y atiende el siguiente job de la cola.

Método	Caso de Error	Comportamiento
Newton-Raphson	$f'(x) \approx 0$	División por cero → job FAILED con mensaje descriptivo
Newton-Raphson	Derivada omitida	Se calcula automáticamente con SymPy
Secante	$f(x_0) \approx f(x_1)$	División por cero → job FAILED
Müller	Discriminante < 0	Se manejan raíces complejas, resultado incluye parte imaginaria
Gauss / Gauss- Jordan	Matriz singular	Error descriptivo → job FAILED
Gauss-Seidel	Diagonal con ceros	No convergencia → job FAILED con aviso
Todos	maxIter alcanzado	Status DONE con converged = false

Todos	Expresión inválida	Error de parsing → job FAILED con mensaje de error
Todos	Job DONE / RUNNING	Idempotencia: no se re-ejecuta

10. Estructura del Repositorio

El repositorio en GitHub (<https://github.com/JL-RZ21/proyectoFinalMetodosNumericos>) sigue la siguiente estructura de carpetas:

proyectoFinalMetodosNumericos/

- api-dotnet/ → API .NET 10 (Controllers, Data, DTOs, Models, Services, Migrations, Dockerfile)
- worker-python/ → Worker Python (methods/, config.py, db.py, redis_client.py, dispatcher.py, main.py, Dockerfile)
- frontEnd/ → SPA React (src/components, pages, utils, api.ts, types.ts, App.tsx, Dockerfile, nginx.conf)
- sql-init/ → Scripts SQL de inicialización del esquema
- docs/ → Documentación (manual-tecnico.pdf, manual- usuario.pdf, informe-final.pdf)
- docker-compose.yml → Orquestación de los cinco contenedores
- .env.example → Plantilla de variables de entorno
- .gitignore → Reglas de exclusión de archivos
- README.md → Instrucciones de despliegue

10.1 Guía paso a paso: levantar el sistema con Docker

10.2 Requisitos previos

- Sistema operativo: Windows 10/11, macOS
- Docker Desktop instalado y corriendo
- Docker Compose
- Navegador moderno: Chrome, Firefox, Edge etc:

10.3 Instalar Docker Desktop

1. Ve a <https://www.docker.com/products/docker-desktop>
2. Descarga la versión para tu sistema operativo e instala.
3. Una vez instalado, abre una terminal y verifica:

```
docker --versión
Docker version 24.0.0, build

docker compose versión
```

```
Docker Compose version v2.20.0
```

10.4 Descargar el proyecto

```
Opción 1: Clonar con Git
git clone https://github.com/JL-RZ21/proyectoFinalMetodosNumericos
cd metodos-numericos

Opción 2: Descargar el .zip desde GitHub y descomprimir
```

10.5 Levantar todos los servicios

Desde la carpeta raíz del proyecto, ejecuta:

```
Levantar (primera vez puede tardar ~2-5 min descargando imágenes)
docker compose up -build

O en segundo plano (sin bloquear la terminal)
docker compose up -d -build
```

Cuando veas los logs de los servicios activos, abre tu navegador en:

☒ **La plataforma está lista**

Abre tu navegador en: <http://localhost:5173>
Deberás ver el título: Plataforma de Métodos Numéricos
El indicador verde 'API online' confirma que el backend responde.

10.6 Detener la plataforma

```
docker compose down

Para eliminar volumen de datos también:
docker compose down -v
```